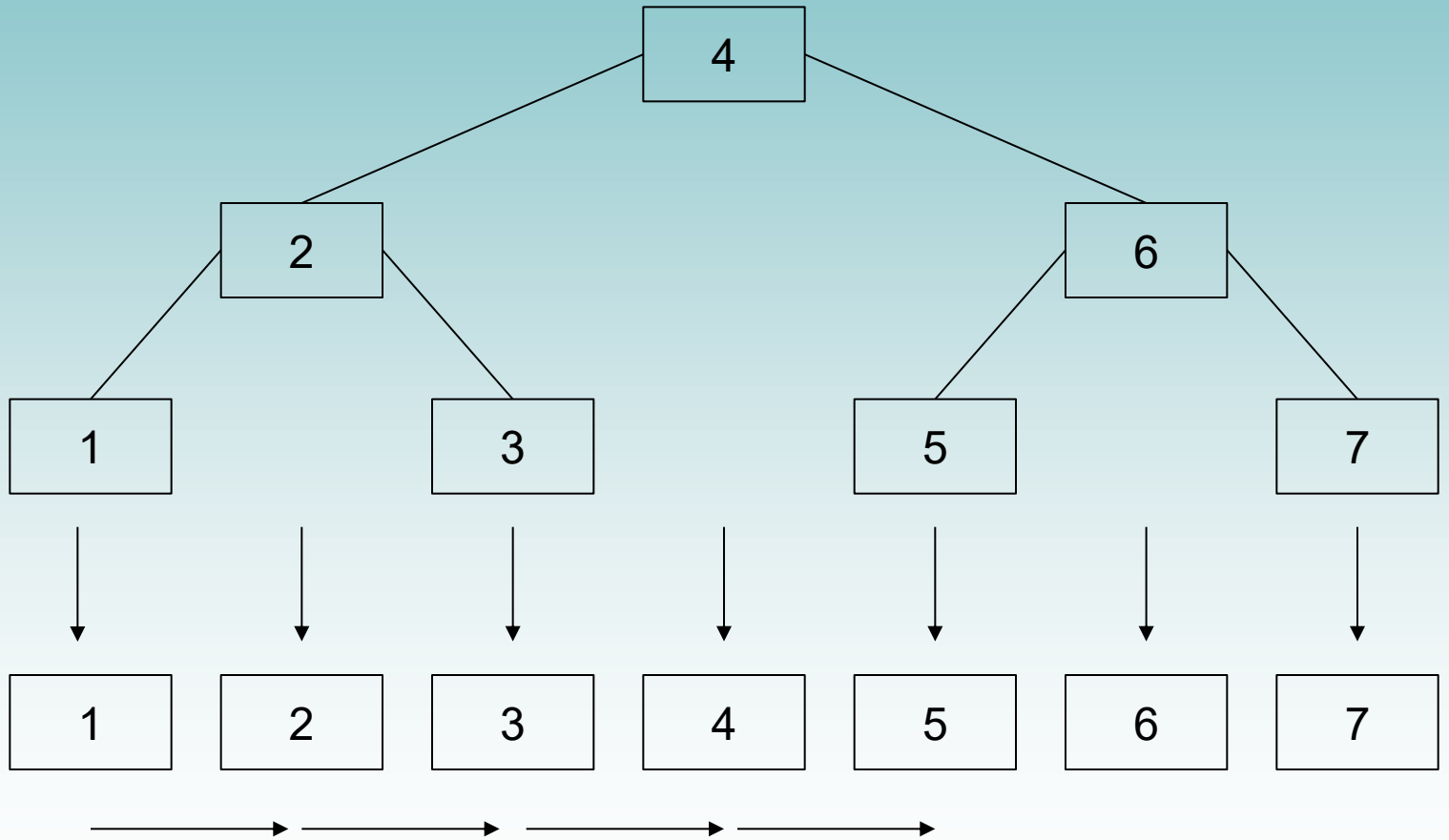
Индексация в СУБД



Понятие индекса

Основная проблема в СУБД – это поиск нужных данных за минимальное время.

- Быстрый поиск может быть выполнен в случае, если данные отсортированы.
- Данные обычно добавляются в конец таблицы.
- Сортировка данных в таблице невозможна, т.к. критериев поиска может быть несколько.

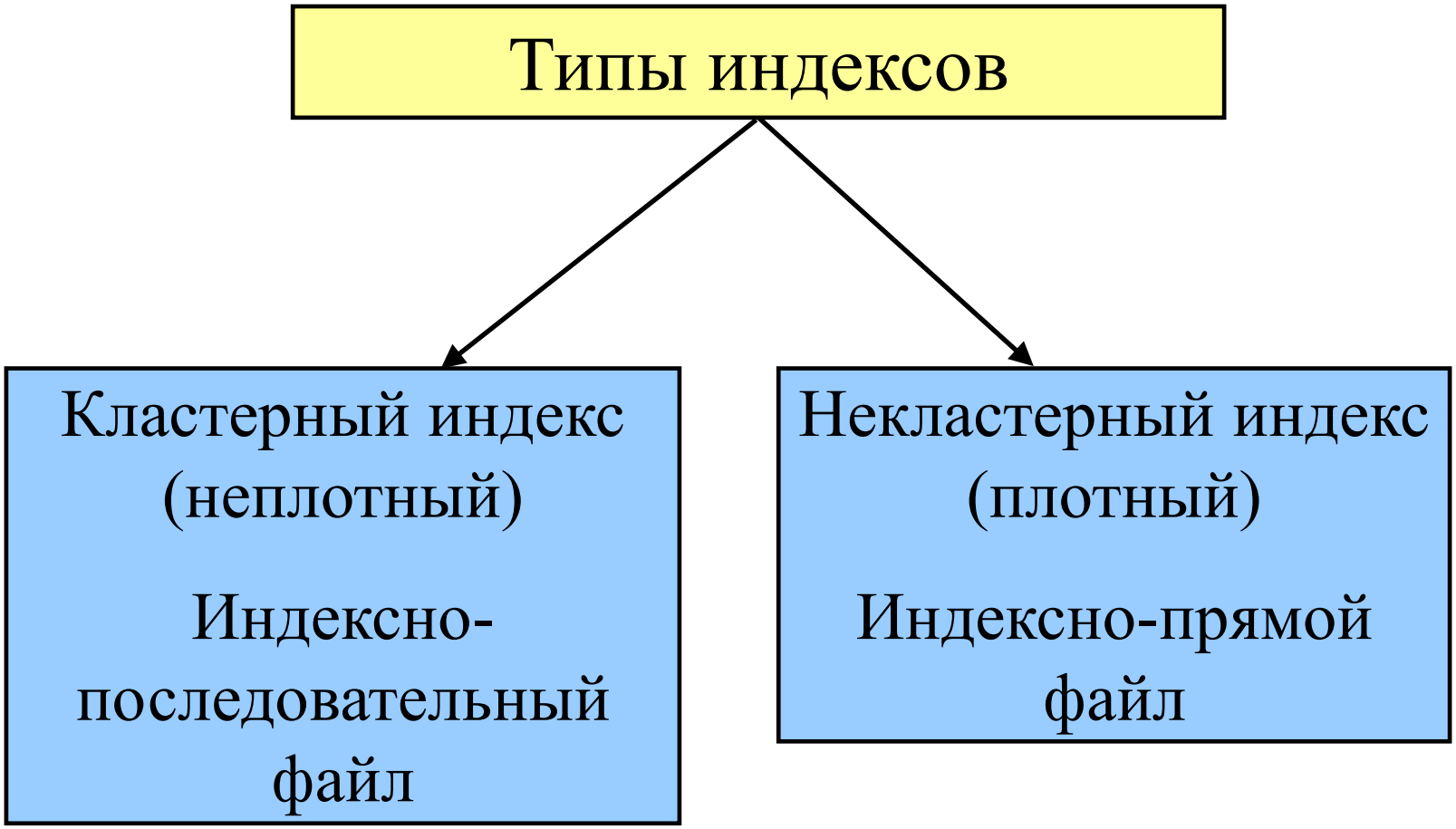
Поэтому для таблицы с данными создаются специальные таблицы для каждого критерия поиска, которые называются ИНДЕКСЫ.

Индекс – это структура данных для быстрого поиска записей в таблице по значению ключа.

Для чего нужна индексация?

- для быстрого поиска строк по условию WHERE,
- для уменьшения числа отбираемых строк (выбирается индекс, по которому отсеивается максимальное число строк),
- для получения строк из других таблиц во время объединений (JOIN),
- для поиска минимального/максимального (MIN() или MAX()) значения в столбце (эти константы сохраняются при индексировании),
- для быстрой сортировки/группировки таблицы по проиндексированным столбцам.

Методы организации индекса



Основная таблица

КодСотр	Фамилия	отдел	Адрес
73	Иванов	04	г.Минск …
75	Степанов	04	г.Минск …
80	Кустов	21	г.Минск …
66	Чернов	22	г.Минск …
06	Трусов	14	г.Минск …
03	Терпухов	12	г.Минск …
83	Павлов	04	г.Минск …
90	Мухина	12	г.Минск …
91	Артемьев	21	г.Минск …
63	Васильев	22	г.Минск …
84	Васильева	04	г.Минск …
81	Уваров	04	г.Минск …
98	Шишкин	21	г.Минск …
92	Удалов	22	г.Минск …

Кластерный индекс

Таблица индекса

Содержит индексные записи в отсортированном порядке, расположенные в блоках.

Основная таблица

Содержит последовательность записей одинаковой длины в отсортированном порядке по индексному ключу, расположенные в блоках, имеющих изначально свободное пространство.

Индексная запись

Индексный ключ
Номер блока в основном файле

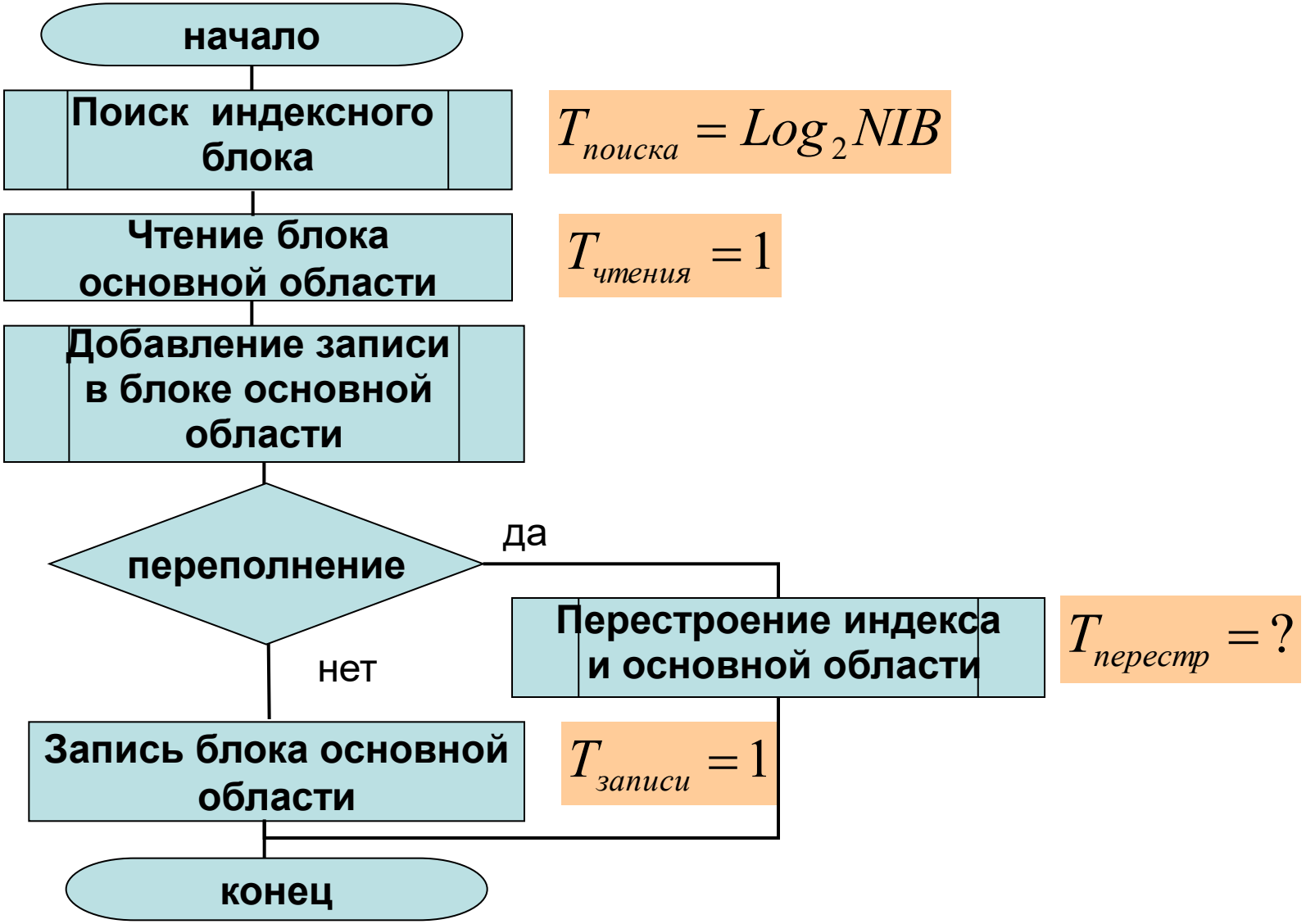
Поиск данных осуществляется по индексному ключу в индексном файле методом бинарного поиска по блокам.

Кластерный индекс



Кластерный индекс

Алгоритм добавления записи



Кластерный индекс

Алгоритм удаления записи



$$T_{поиска} = Log_2 NIB$$

$$T_{чтения} = 1$$

$$T_{записи} = 1$$

Кластерный индекс

Оценка времени на выполнение основных операций

(в максимальном количестве обращений к диску)

$$T_{поиска} = Log_2(\frac{N \times LZ \times LI}{LB^2}) + 1$$

$$T_{добав} = T_{поиск} + 1$$

$$T_{удал} = T_{поиск} + 1$$

N – количество записей в основной таблице;

LB – размер блока;

LZ- длина записи основной таблицы

LI– длина записи в индексной таблице;

Некластерный индекс

Таблица индекса

Содержит индексные записи в отсортированном порядке, расположенные **в блоках**, имеющих изначально свободное пространство.

Основная таблица

Содержит последовательность записей одинаковой длины в произвольном порядке.

Индексная запись

Индексный ключ
Номер записи в основном файле

Поиск данных осуществляется по индексному ключу в индексной таблице методом бинарного поиска по блокам.

Некластерный индекс

Таблица индекса

	КодСотр	Номер записи
Блок 1	03	6
	06	5
	63	10
	66	4
	73	1
Блок 2	75	2
	80	3
	81	12
	83	7
Блок 3	84	11
	90	8
	91	9
Блок 4	92	14
	98	13

Основная таблица

КодСотр	Фамилия	отдел	Адрес
73	Иванов	04	г.Минск …
75	Степанов	04	г.Минск …
80	Кустов	21	г.Минск …
66	Чернов	22	г.Минск …
06	Трусов	14	г.Минск …
03	Терпухов	12	г.Минск …
83	Павлов	04	г.Минск …
90	Мухина	12	г.Минск …
91	Артемьев	21	г.Минск …
63	Васильев	22	г.Минск …
84	Васильева	04	г.Минск …
81	Уваров	04	г.Минск …
98	Шишкин	21	г.Минск …
92	Удалов	22	г.Минск …

Некластерный индекс

2-я таблица индекса

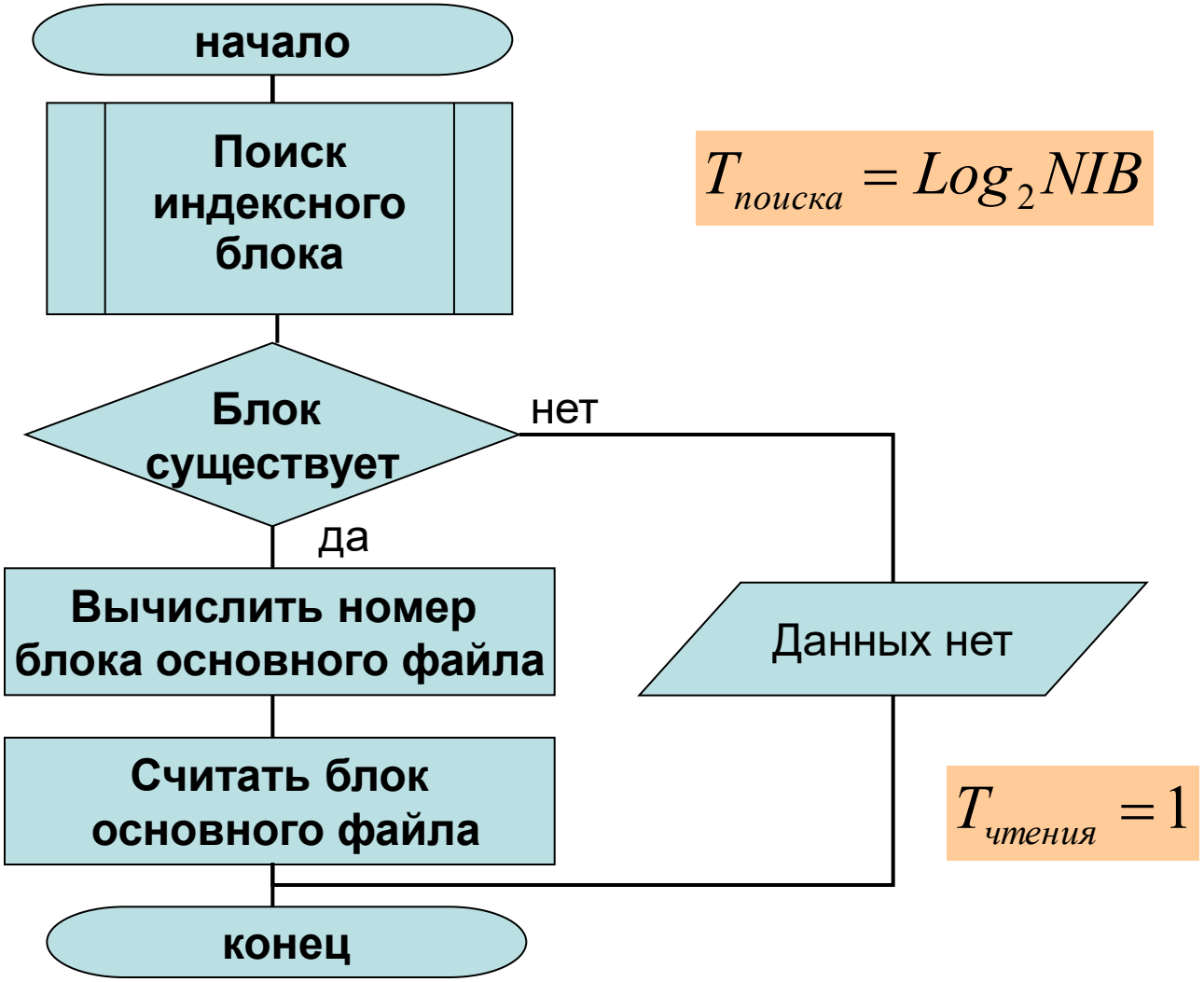
	Фамилия	Номер записи
Блок 1	Артемьев	9
	Васильев	10
	Васильева	11
	Иванов	1
Блок 2	Кустов	3
	Мухина	8
	Павлов	7
	Степанов	2
	Терпухов	6
Блок 3	Трусов	5
	Уваров	12
	Удалов	14
	Чернов	4
	Шишкин	13

Основная таблица

КодСотр	Фамилия	отдел	Адрес
73	Иванов	04	г.Минск …
75	Степанов	04	г.Минск …
80	Кустов	21	г.Минск …
66	Чернов	22	г.Минск …
06	Трусов	14	г.Минск …
03	Терпухов	12	г.Минск …
83	Павлов	04	г.Минск …
90	Мухина	12	г.Минск …
91	Артемьев	21	г.Минск …
63	Васильев	22	г.Минск …
84	Васильева	04	г.Минск …
81	Уваров	04	г.Минск …
98	Шишкин	21	г.Минск …
92	Удалов	22	г.Минск …

Некластерный индекс

Алгоритм поиска данных

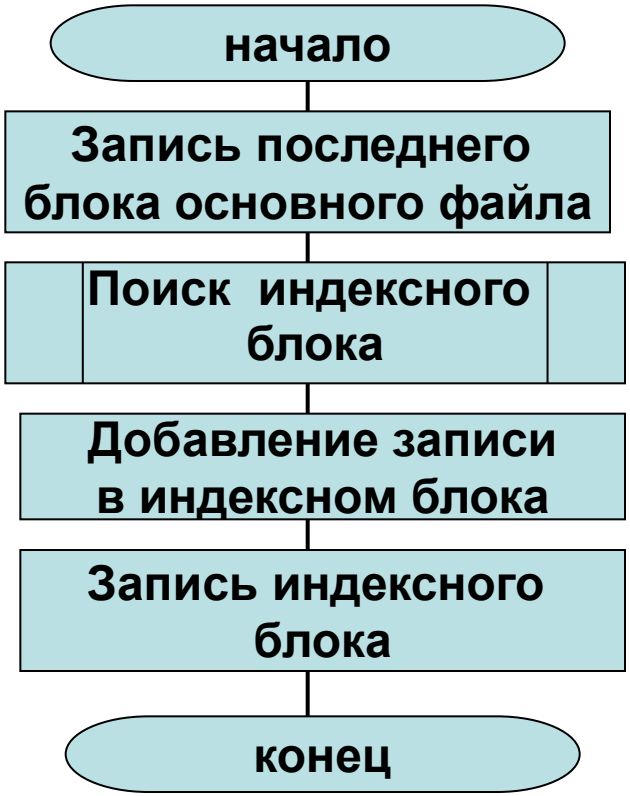


$$T_{поиска} = Log_2 NIB$$

$$T_{чтения} = 1$$

Некластерный индекс

Алгоритм добавления записи



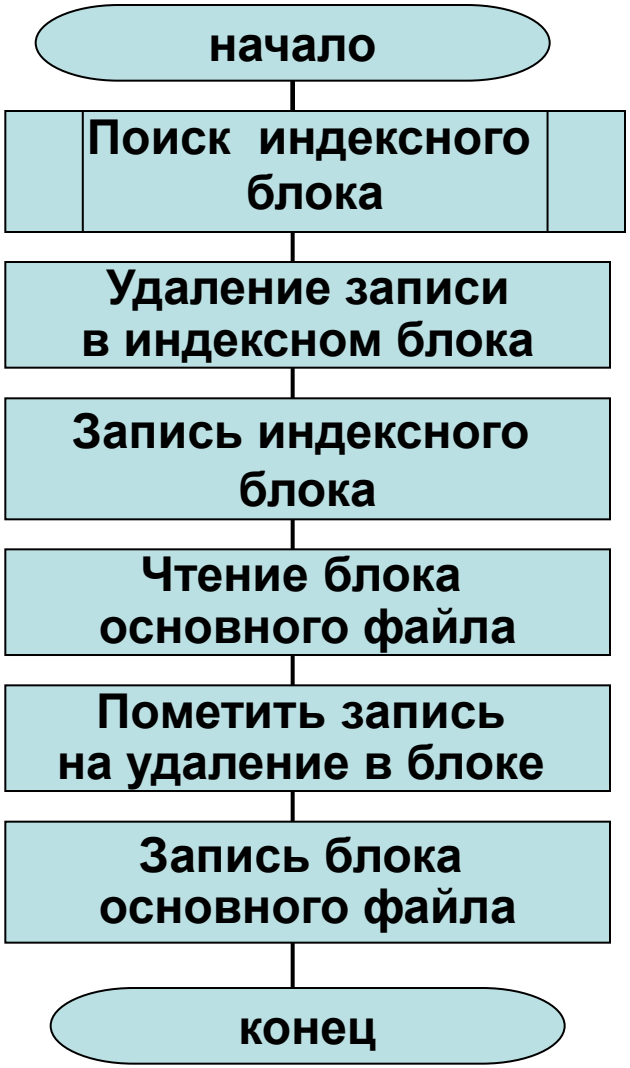
$$T_{записи} = 1$$

$$T_{поиска} = Log_2 NIB$$

$$T_{записи} = 1$$

Некластерный индекс

Алгоритм удаления записи



$$T_{поиска} = Log_2 NIB$$

$$T_{записи} = 1$$

$$T_{чтения} = 1$$

$$T_{записи} = 1$$

Некластерный индекс

Оценка времени на выполнение основных операций (в максимальном количестве обращений к диску)

без использования индекса

$$T_{поиска} = \frac{N}{LB / LZ}$$

$$T_{добав} = 1$$

$$T_{удал} = T_{поиск} + 1$$

с использованием индекса

$$T_{поиска} = Log_2(\frac{N}{LB / LI}) + 1$$

$$T_{добав} = T_{поиск} + 1$$

$$T_{удал} = T_{поиск} + 2$$

N – количество записей в основной таблице;

LB – размер блока;

LZ- длина записи основной таблицы

LI– длина записи в индексной таблице;

Сравнение кластерного и некластерного индексов

Критерий	*Кластеризованный*	*Некластеризованный*
Порядок хранения	Сортирует и хранит данные по правилу сортировки; использует **первичный ключ** для структуризации данных в таблице; всегда изменяет физический порядок хранимых данных	Не упорядочивает физическое хранение, используя метки для доступа к данным; создает свою **отдельную структуру** данных, которая не зависит от фактической таблицы, для которой он строится
Конечные листья индекса	Используются для хранения	Не используются для хранения

Сравнение кластерного и некластерного индексов

Критерий	*Кластеризованный*	*Некластеризованный*
Место на диске	Занимает много места	Занимает мало места
Скорость доступа	Быстрый	Медленный
Доп. место на диске	Не нужно	Необходимо, так как индекс хранится отдельно от физических данных таблицы
Простота использования	Не требует явного объявления и создается по умолчанию при определении ключа, то есть в принципе легче в использовании — в том числе в плане синтаксиса **КРОМЕ PostgreSQL**	Требует явного определения; содержит в себе только те столбцы таблицы, по которым определен, поэтому системе запросов нужна дополнительная операция для того, чтобы получить данные из индекса

Сравнение кластерного и некластерного индексов

Критерий	*Кластеризованный*	*Некластеризованный*
Сортировка данных	Возможна	Невозможна
Создание нескольких индексов на одну таблицу	Невозможно	Возможно
Специфика	Может повысить производительность при извлечении требуемых данных	Применяется только к неключевым столбцам, которые используются в запросах join

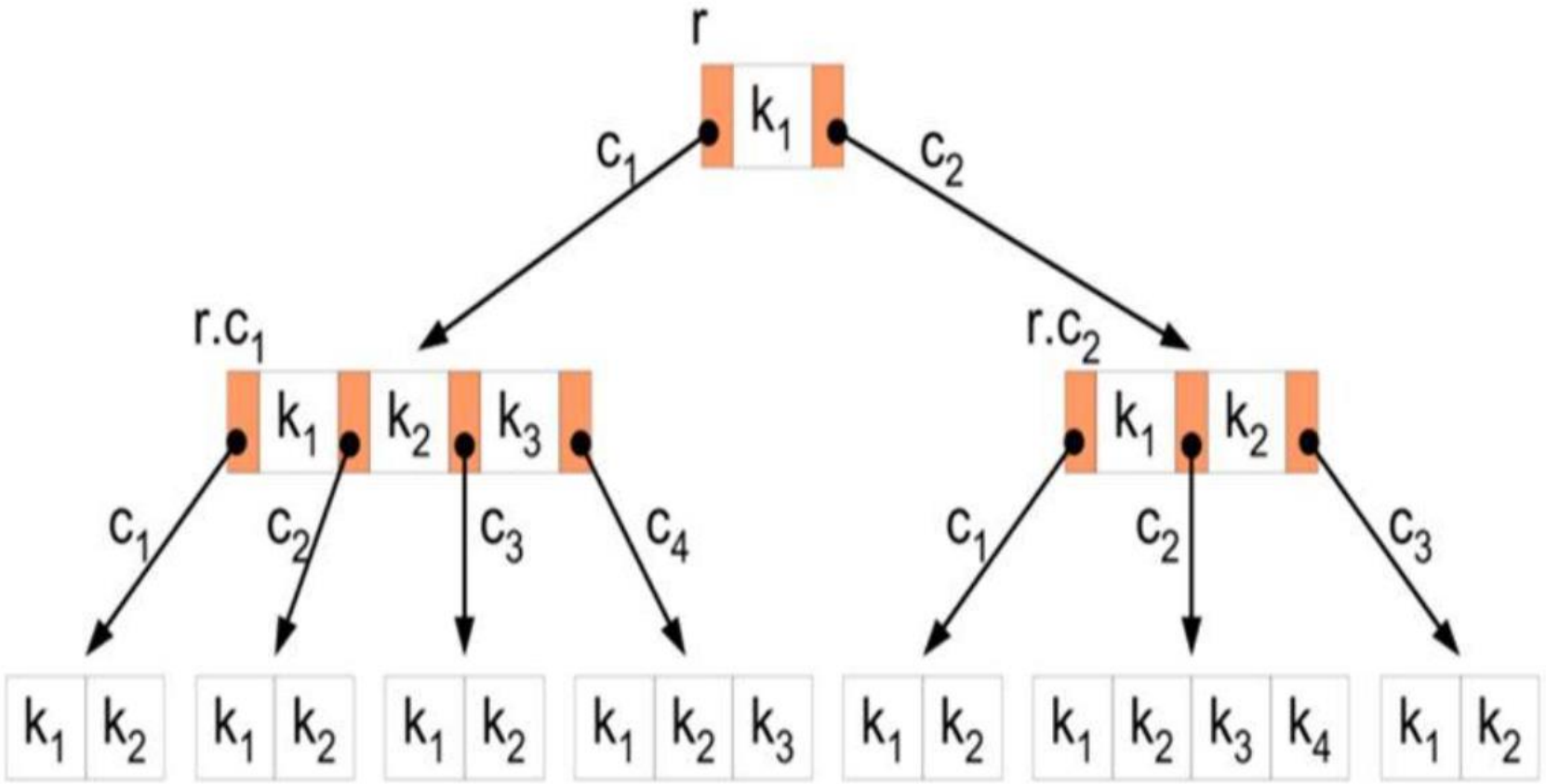
Индекс первичного ключа в PostgreSQL

- В PostgreSQL индекс первичного ключа по умолчанию не является кластерным.
- Может стать таковым после выполнения команды CLUSTER, которая физически упорядочивает данные таблицы в соответствии с выбранным индексом.
- Это единственный способ сделать первичный ключ кластерным в PostgreSQL, так как система сама по себе не создает кластерные индексы автоматически при создании первичного ключа.

```
CREATE TABLE users (
id SERIAL PRIMARY KEY,
name VARCHAR(100));

CLUSTER users USING users_pkey;
```

B-tree index


r
k1
c1
c2
r.c1
k1
k2
k3
r.c2
k1
k2
c1
c2
c3
c4
c1
c2
c3
k1 k2
k1 k2
k1 k2
k1 k2 k3
k1 k2
k1 k2 k3 k4
k1 k2

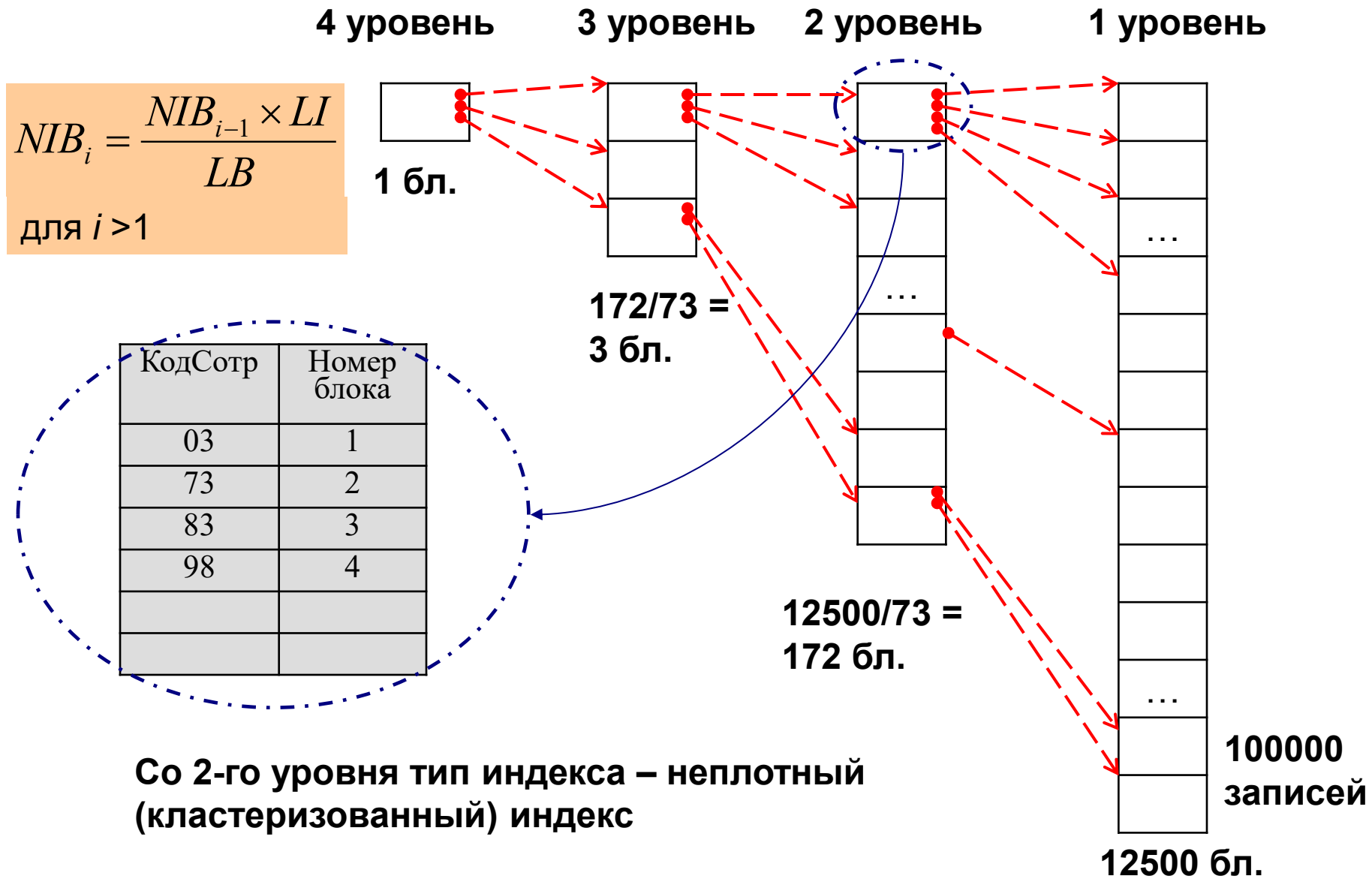
- **B-tree – сбалансированное сильноветвящееся дерево во внешней памяти;**

- **Сбалансированность означает, что длины пути от корня дерева к любому его листу одинаковы (все листья на одном уровне);**

- **Ветвистость дерева – свойство каждого узла дерева ссылаться на большое число узлов-потомков;**

- **Поиск в B-дереве – это прохождение от корня к листу в соответствии с заданным значением ключа.**

B-tree index


4 уровень
3 уровень
2 уровень
1 уровень
$NIB_i = \frac{NIB_{i-1} \times LI}{LB}$
для i >1
1 бл.
172/73 =
3 бл.
12500/73 =
172 бл.
КодСотр
Номер блока
03
1
73
2
83
3
98
4
100000 записей
12500 бл.
Со 2-го уровня тип индекса – неплотный (кластеризованный) индекс

B-tree index

Оценка времени на выполнение основных операций (в максимальном количестве обращений к диску)

$$T_{поиска} = NLI$$

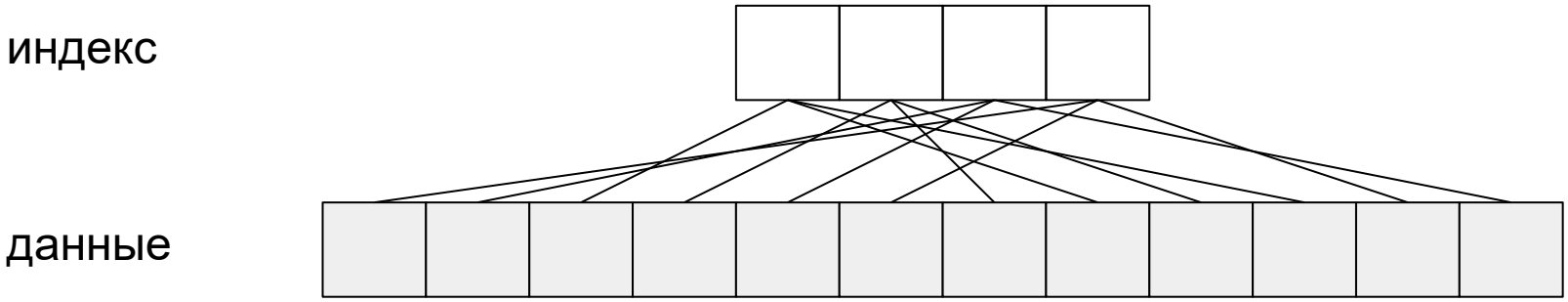
$$T_{добав} = T_{поиск} + 1$$

$$T_{удал} = T_{поиск} + 1$$

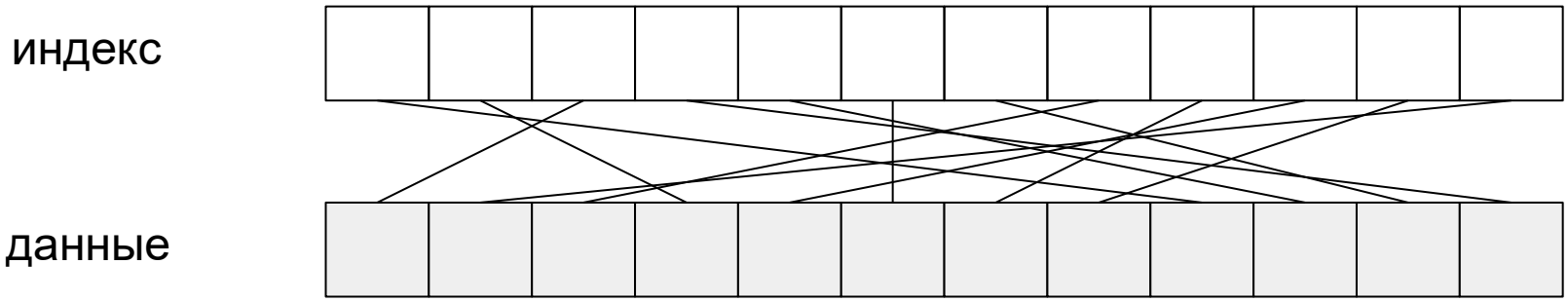
где ***NLI*** **– количество уровней индексов.**

HASH index

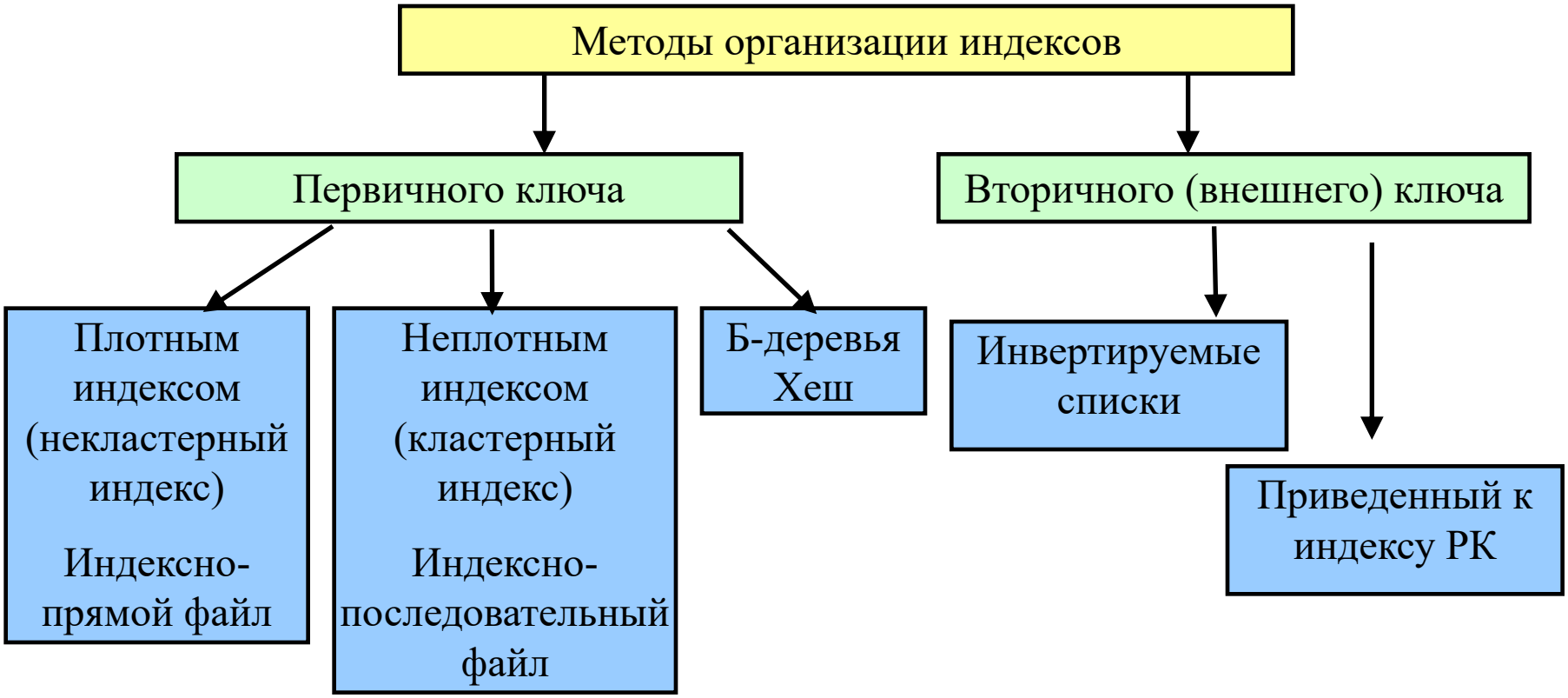
Обычный индекс



Уникальный индекс



Методы организации индекса



Основная таблица

КодСотр	Фамилия	отдел	Адрес
73	Иванов	04	г.Минск …
75	Степанов	04	г.Минск …
80	Кустов	21	г.Минск …
66	Чернов	22	г.Минск …
06	Трусов	14	г.Минск …
03	Терпухов	12	г.Минск …
83	Павлов	04	г.Минск …
90	Мухина	12	г.Минск …
91	Артемьев	21	г.Минск …
63	Васильев	22	г.Минск …
84	Васильева	04	г.Минск …
81	Уваров	04	г.Минск …
98	Шишкин	21	г.Минск …
92	Удалов	22	г.Минск …

Инвертируемые списки

Индексная таблица 1-го уровня

Содержит индексные записи в отсортированном порядке, расположенные в блоках, имеющих изначально свободное пространство.

Индексная таблица 2-го уровня

Содержит цепочку номеров записей в отсортированном порядке, расположенных в блоках.

Основная таблица

Содержит последовательность записей одинаковой длины в произвольном порядке.

Индексная запись

Индексный ключ
Номер блока в индексном файле 2-го уровня

Индексная запись

Номер блока в основном файле

Поиск данных осуществляется по индексному ключу в индексной таблице (файле)1-го уровня методом бинарного поиска по блокам, с дальнейшим переходом к цепочке записей, расположенных в индексной таблице (файле) 2-го уровня.

Инвертируемые списки



Ограничения индексации

Пример 1:

SELECT first_name FROM people WHERE last_name = ‘Петров’;

Для выполнения такого запроса без индекса СУБД должна проверить поле last_name в каждой строке таблицы (полный перебор значений).

При использовании индекса СУБД просто проходит по B-дереву, пока не найдёт запись «Петров».

Пример 2:

SELECT email_address FROM customers WHERE email_address LIKE ‘%@gmail.com’;

Индексы строятся в предположении, что слова/символы идут слева направо. Использование символа подстановки в начале условия поиска исключает для СУБД возможность использования поиска по B-дереву (всё равно придется сканировать всю таблицу).

Ограничения индексации

Во многих СУБД эта проблема может быть решена созданием дополнительного индекса по выражению reverse(email_address) и формированием запроса вида:

SELECT email_address FROM customers WHERE reverse(email_address) LIKE reverse('%@ google.com');

В данном случае символ подстановки окажется в самой правой позиции (moc.elgoog@%), что не исключает использование индекса по reverse(email_address).

Рекомендации по созданию индексов

Факторы, определяющие «хороший» индекс

- число столбцов в индексе не более 4-5

- в индекс включаются не часто обновляемые столбцы

- создание индексов по FK

- создание индексов по столбцам из выражения WHERE часто выполняемых запросов

- (не использовать поиски с предикатом LIKE)

Факторы, определяющие «плохой» индекс

- по столбцам, входящих в индекс, не выполняются поиски строк

- слишком много столбцов в индексе

- слишком мало записей в индексируемой таблице

- слишком много индексов

Операторы создания и удаления индексов

CREATE [UNIQUE] INDEX [CONCURRENTLY] [[IF NOT EXISTS] ***имя***]
ON [ONLY] ***имя_таблицы*** [USING ***метод***]

({ ***имя_столбца*** | (***выражение***) } [COLLATE ***правило_сортировки***] [***класс_операторов*** [(***параметр_класса_оп*** = ***значение*** [, ...])]] [ASC | DESC] [NULLS { FIRST | LAST }] [, ...])

[INCLUDE (***имя_столбца*** [, ...])]

[NULLS [NOT] DISTINCT]

[WITH (***параметр_хранения*** [= ***значение***] [, ...])]

[TABLESPACE ***табл_пространство***]

[WHERE ***предикат***]

DROP INDEX ***имя*** ON ***имя_таблицы***;